

Logbuch – Florian Adamczyk

Matrikelnummer: 8105234

Kurs: KI 1 – Projekt, WS 2025/26

Gruppe: 308

Eintrag 1 – Recherche: Git-Workflow & nbstripout

Datum: 11.02.2026

Zeitraum: ca. 16:00 – 20:00 Uhr (Selbststudium, im Anschluss an Gruppentreffen 1)

Gesamtzeit: ca. 4 h

Arbeitsschritte

- Im Gruppentreffen 1 (11.02.) wurde u. a. die Nutzung von Git/GitHub beschlossen. Im Anschluss habe ich eigenständig recherchiert, wie man Jupyter Notebooks sinnvoll versionieren kann. (*ca. 30 min*)
- **Problem identifiziert:** Notebook-Outputs und Metadaten erzeugen unnötige Merge-Konflikte und erschweren Code-Reviews erheblich. Lektüre von Blogposts und Stack-Overflow-Threads zu Jupyter + Git. (*ca. 45 min*)
- **Lösung gefunden: nbstripout** — ein Tool, das Zell-Outputs automatisch vor jedem Commit entfernt (Git-Hook). Offizielle Dokumentation und GitHub-Repo gelesen, mehrere Alternativen verglichen (**nbconvert --clear-output**, manuelle Pre-Commit-Hooks). (*ca. 45 min*)
- GitHub-Repository initial eingerichtet (**git init**, Remote angelegt, README aus dem GitLab-Kursrepo übernommen). (*ca. 30 min*)
- **nbstripout** lokal installiert und als Git-Hook ausprobiert; initialer Test-commit mit einem Notebook erstellt und Diff auf saubere Ausgabe geprüft. (*ca. 30 min*)
- Dokumentation in **Ressources.md** erstellt: Installationsanleitung, Einrichtung als Git-Hook, Links zu den relevanten Repositories (Arbeitsrepo, Kurs-GitLab, Vorlesungsfolien, Lösungen). (*ca. 60 min*)

Entscheidungen

Entscheidung	Begründung
nbstripout als verpflichtender Git-Hook	Vermeidet Merge-Konflikte durch Notebook-Outputs
Zentrale Dokumentation in Ressources.md	Alle Teammitglieder können Installationsschritte dort nachschlagen

Schwierigkeiten

- Keine direkten Schwierigkeiten; die Recherche war erfolgreich.
-

Eintrag 2 – Erste Repository-Strukturierung

Datum: 17.02.2026

Zeitraum: Im Anschluss an Gruppentreffen 2, ca. 3 h

Gesamtzeit: ca. 3 h

Arbeitsschritte

- Wie am Gruppentreffen 2 am 17.02. besprochen: **nbstripout** im Team als Standard vorgestellt; Fragen der anderen Teammitglieder zur Installation beantwortet. (*ca. 30 min*)
- PDFs/-Ordner im Repository angelegt, Aufgabenstellung (PDF) heruntergeladen und eingesehen. (*ca. 20 min*)
- Alte Test-Dateien und überflüssige Artefakte aus der initialen Repository-Einrichtung aufgeräumt (Branches bereinigt, überflüssige Commits analysiert). (*ca. 45 min*)
- `.gitattributes` konfiguriert, um Notebooks korrekt mit **nbstripout** zu behandeln; Konfiguration durch einen Test-Commit verifiziert. (*ca. 45 min*)
- **Meine Aufgabe bis zum nächsten Treffen** (wie am Gruppentreffen besprochen): Datenbereinigung und Datenanalyse gemeinsam mit Kolja — erste Planung der Vorgehensweise notiert. (*ca. 40 min*)

Schwierigkeiten

- Keine nennenswerten Probleme.
-

Eintrag 3 – Projektstruktur, Utils-Modul, Notebooks & erste Modelle

Datum: 20.02.2026

Zeitraum: ca. 07:00 – 22:00 Uhr (ganztagig, inkl. Gruppentreffen 3 von 9:00–11:30 Uhr)

Gesamtzeit: ca. 35 h (*verteilt auf 20.02. und nachfolgende Abende bis 25.02.*)

Dies war der zentrale Arbeitsblock, in dem ich die gesamte Projektgrundstruktur aufgebaut habe. Ausgangspunkt waren die Aufgaben, die am Gruppentreffen 2 (17.02.) vereinbart wurden: Datenbereinigung und Datenanalyse. Im Zuge dessen habe ich über die reine Datenbereinigung hinaus einen vollständigen

Masterplan sowie die gemeinsame Code-Infrastruktur für das gesamte Projekt erstellt.

Arbeitsschritte

0. Gruppentreffen 3 (*ca. 2,5 h*)

- Teilnahme am Gruppentreffen 3 (09:00–11:30 Uhr): Präsentation des bisherigen Stands, Diskussion der Aufgabenverteilung P1–P5, Abstimmung über Projektstruktur und Zeitplan. (*ca. 2,5 h*)

1. Masterplan und Aufgabenverteilung (*ca. 3 h*)

- **Masterplan.md** geschrieben: Projektphasen (Setup → EDA/Baseline → Individuelle Methoden → Vertiefung → Portfolio), Zuordnung der Methoden zu fünf Personen (P1–P5), Bewertungshinweise, Projektstruktur. (*ca. 90 min*)
- **Aufgabenübersicht.md** erstellt: Für jede der fünf Aufgaben eine kurze Beschreibung (Was macht man? Schwierigkeiten? Schwierigkeitsgrad?), damit sich die Teammitglieder orientiert eine Aufgabe auswählen können. (*ca. 60 min*)
- Die Idee hinter der Aufteilung: Jede Person bearbeitet einen eigenen methodischen Schwerpunkt (Regularisierung, Decision Trees, Ensemble, kNN, Neuronales Netz), sodass sich die individuellen Logbücher inhaltlich unterscheiden — wichtig für die 40 %-Bewertung der Logbuch-Zusammenfassung laut Aufgabenstellung. Konzept schriftlich ausgearbeitet und im Masterplan dokumentiert. (*ca. 30 min*)

2. Python-Umgebung & TensorFlow (*ca. 3 h*)

- **Python-Versionsproblem entdeckt:** Am Gruppentreffen 1 hatten wir Python 3.14.3 vereinbart. Beim Versuch, TensorFlow zu installieren, scheiterte `pip install tensorflow` — es existieren keine Wheels für Python 3.14. TensorFlow unterstützt offiziell bis Python 3.13 (ab TF 2.18). Fehleranalyse und Recherche der offiziellen TF-Kompatibilitätsmatrix. (*ca. 60 min*)
- **Lösung:** Auf meinem System (Arch Linux) war `python3.13` bereits installiert. Virtuelle Umgebung neu erstellt:

```
python3.13 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```


(*ca. 30 min*)
- TensorFlow 2.20.0 erfolgreich installiert und mit einem kurzen Testskript verifiziert (`tf.__version__`, einfaches Dense-Modell). (*ca. 30 min*)

- **requirements.txt** erstellt mit allen benötigten Paketen: `numpy`, `pandas`, `matplotlib`, `seaborn`, `scikit-learn`, `tensorflow`, `jupyter`, `ipykernel`, `nbstripout`. Versionen recherchiert und auf Kompatibilität geprüft. (ca. 60 min)

3. Gemeinsames Utils-Modul (utils/) (ca. 12 h) Damit alle Teammitglieder auf identischen Daten und einheitlichen Metriken arbeiten, habe ich ein zentrales Python-Modul erstellt:

- **utils/data.py** — Laden des California-Housing-Datensatzes, Datenbereinigung (Cut-offs bei `MedHouseVal` = 5.001 und `HouseAge` = 52, Ausreißer über 98. Perzentil bei `AveRooms`, `AveBedrms`, `Population`, `AveOccup`), standardisierter Train/Test-Split (80/20, `random_state=42`), optionale Skalierung (MinMax / Standard). (ca. 4 h)
 - Die Bereinigungsschritte orientieren sich an den Vorlesungsunterlagen (Blatt 9/10 Lösung). Lösung nachvollzogen, Cut-off-Werte reproduziert und im Code kommentiert. (in obigen 4 h enthalten)
- **utils/evaluation.py** — Einheitliche Auswertung: R^2 , MAE, RMSE für Train- und Testdaten. Globale Ergebnis-Sammlung (`add_result()`, `compare_models()`), damit am Ende eine gemeinsame Vergleichstabelle über alle Modelle entsteht. (ca. 2 h)
- **utils/plotting.py** — Standardisierte Visualisierungen: Predicted-vs-Actual-Plot, Residuenplot, Feature-Importances, Korrelations-Heatmap, Histogramme, Features-vs-Target-Scatterplots. Alle Plots werden automatisch als PNG und PDF nach `results/` exportiert (wie in der Aufgabenstellung gefordert: keine Screenshots). Design und Formatierung iterativ angepasst. (ca. 5 h)
- **utils/__init__.py** — Re-Export der wichtigsten Funktionen für einfachen Import in den Notebooks. Testen aller Importe in einem Scratch-Notebook. (ca. 60 min)

4. Notebooks erstellt (ca. 8 h) Acht Notebooks angelegt, die den Phasen des Masterplans entsprechen. Für jedes Notebook: Grundstruktur aufgebaut, Imports eingerichtet, `sys.path`-Eintrag gesetzt, Datenladen-Zelle vorbereitet, Platzhalter-Abschnitte für Methodik und Auswertung angelegt, abschließende Ergebniszelle mit `add_result()`-Aufruf vorbereitet. (ca. 1 h pro Notebook)

Notebook	Inhalt
0a_EDA.ipynb	Explorative Datenanalyse: Histogramme, Scatterplots, Korrelationsheatmap (roh und bereinigt)
0b_Baseline_Lineare_Regression.ipynb	Lineare Regression als Referenzmodell (alle Features, nur MedInc), Pred-vs-Actual, Residuen

Notebook	Inhalt
0c_Scaling_Data.ipynb	Vergleich verschiedener Skalierungsmethoden (MinMax, Standard) mit Visualisierung
01_LASSO_Ridge.ipynb	P1: LassoCV/RidgeCV mit/ohne Skalierung, polynomiale Features
02_Decision_Tree.ipynb	P2: Entscheidungsbaum, Depth-Tuning, Pruning, Feature Importances
03_Ensemble.ipynb	P3: Random Forest, Gradient Boosting, GridSearchCV
04_kNN_Regression.ipynb	P4: kNN-Regression, Skalierungseffekt, k-Optimierung
05_Neural_Network.ipynb	P5: TensorFlow/Keras NN, verschiedene Architekturen, Vergleich mit LR

- Jedes Notebook importiert die gemeinsamen Funktionen aus `utils/` und arbeitet auf demselben bereinigten Datensatz mit identischem Split. (*in obigen 8 h enthalten*)
- Die Notebooks sind inhaltlich so vorbereitet, dass jedes Teammitglied direkt mit seinem Schwerpunkt starten kann — die Grundstruktur (Daten laden, Ergebnis speichern, Plots exportieren) ist in jedem Notebook bereits angelegt. (*in obigen 8 h enthalten*)

5. EDA & Baseline durchgeführt (*ca. 5 h*)

- **EDA (0a_EDA.ipynb):** Rohdaten untersucht (20.640 Samples, 8 Features, keine fehlenden Werte), Histogramme vor und nach Bereinigung erstellt, Korrelationsheatmap zeigt stärkste Korrelation zwischen `MedInc` und `MedHouseVal`. Nach Bereinigung verbleiben ca. 17.386 Samples. Plots exportiert und Interpretation schriftlich festgehalten. (*ca. 3 h*)
- **Baseline (0b_Baseline_Lineare_Regression.ipynb):** Lineare Regression auf allen 8 Features: **R^2 Test 0.633**, MAE 0.434, RMSE 0.578. Residuenplot zeigt systematische Muster → nicht-lineare Zusammenhänge, die lineare Regression nicht erfassen kann. Interpretation und Kommentare im Notebook ergänzt. (*ca. 2 h*)

6. nbstripout & .gitignore-Konfiguration (*ca. 2 h*)

- Mehrere Versuche, `nbstripout` und `.gitignore` korrekt zu konfigurieren. `nbstripout` funktionierte zunächst nicht zuverlässig als Git-Hook (Outputs wurden trotzdem committet). Problem lag an fehlender `.gitattributes`-Konfiguration. Debugging und erneute Einrichtung. (*ca. 90 min*)

- `.gitignore` für `results/*.pdf` eingerichtet, dann wieder angepasst, da einige PDFs doch versioniert werden sollten. Mehrere Commits nötig. (*ca. 30 min*)

Entscheidungen

Entscheidung	Begründung
Python 3.13 statt 3.14	TensorFlow-Inkompatibilität mit Python 3.14
TensorFlow 2.20.0	Neueste Version, kompatibel mit Python 3.13
<code>RANDOM_STATE = 42</code> (global)	Reproduzierbare Ergebnisse, identischer Split für alle Teammitglieder
Test-Split 20 %	Standard-Aufteilung, wie in Vorlesung verwendet
Ausreißer-Grenze: 98. Perzentil	Orientierung an der Vorlesung (Blatt 9/10 Lösung)
Getrennte Notebooks pro Methode	Weniger Merge-Konflikte in Git, klare Zuordnung zu Personen
Gemeinsames <code>utils/-</code> Modul	Einheitliche Daten, Metriken und Plots für alle Teammitglieder

Schwierigkeiten

- **TensorFlow + Python 3.14:** Keine kompatiblen Wheels verfügbar. Lösung: Wechsel auf Python 3.13.
- **utils als lokales Modul:** `requirements.txt` enthielt zunächst `utils>=0.1.0` (ein fremdes PyPI-Paket). Entfernt — das lokale `utils/-`Modul wird über den Notebook-internen `sys.path`-Eintrag eingebunden.
- **nbstripout-Konfiguration:** Mehrere Anläufe nötig, bis der Git-Hook zuverlässig funktionierte.
- **.gitignore für PDFs:** Zunächst alle PDFs ignoriert, dann einzelne wieder freigegeben — erforderte mehrere Commits, um die gewünschte Konfiguration zu erreichen.

Offene Fragen

- Aufgabenverteilung P1–P5 steht noch nicht fest; soll im nächsten Gruppentreffen besprochen werden.

Eintrag 4 – Python-Versionskonflikt auf macOS (Intel)

Datum: 20.02.2026

Zeitraum: Während und nach Gruppentreffen 3 (9:00–11:30 Uhr)

Gesamtzeit: ca. 2 h

Arbeitsschritte

- Während des Gruppentreffens 3 am 20.02. hat Kolja versucht, die Umgebung auf seinem Intel-Mac einzurichten. Problem live beobachtet und gemeinsam analysiert. (*ca. 30 min*)
- **Problem:** TensorFlow ab Version 2.17 unterstützt keine Intel-Macs mehr (nur Apple Silicon oder Linux/Windows). Python 3.13 + TensorFlow 2.20 (mein Setup auf Arch Linux) lässt sich auf Koljas Mac nicht installieren. Recherche der offiziellen TF-Release-Notes und PyPI-Paketmetadaten. (*ca. 45 min*)
- Kolja hat als Workaround die `requirements.txt` auf Python 3.11 und TensorFlow 2.16.2 angepasst — diese Version unterstützt noch Intel-Macs. Änderung geprüft und im Repository nachvollzogen. (*ca. 20 min*)
- **Offene Entscheidung:** Es steht noch aus, ob wir einheitlich auf Python 3.11 + TF 2.16 wechseln oder ob unterschiedliche lokale Umgebungen akzeptiert werden. Optionen schriftlich festgehalten und Vor-/Nachteile abgewogen. (*ca. 25 min*)

Schwierigkeiten

- Inkompatibilität von TensorFlow 2.17 mit macOS auf Intel-Prozessoren.
- Falls auf TF 2.16 gewechselt wird, muss geprüft werden, ob alle Notebook-Features kompatibel bleiben.

Nächster Schritt

- Klärung der Python-/TensorFlow-Version im nächsten Gruppentreffen.
- Mögliche Alternativen: JLU JupyterHub für rechenintensive Aufgaben (wurde bereits am Gruppentreffen 2 am 17.02. als Option angesprochen).

Eintrag 5 – Auswertung und derzeitiges Fazit: Neuronales Netz (05_Neural_Network.ipynb)

Datum: 23.02.–26.02.2026

Zeitraum: Selbststudium, verteilt auf mehrere Abende

Gesamtzeit: ca. 11 h

Arbeitsschritte

- Kursunterlagen zu Neuronalen Netzen (Kapitel 3.3, Blatt 11/12) sowie TensorFlow/Keras-Dokumentation zur Auffrischung durchgelesen. (*ca. 90 min*)
- Notebook-Struktur aufgebaut: Abschnitte für Datenloading, Modellarchitektur, Training, Auswertung und Fazit angelegt; Imports und `sys.path`-Eintrag eingerichtet. (*ca. 45 min*)
- Grundmodell (einzelnes Dense-Layer) implementiert, kompiliert und trainiert; erste Metriken abgelesen und mit der Baseline verglichen. (*ca. 60 min*)
- Verschiedene Architekturen (Modell 1–5) systematisch implementiert und getestet: schrittweise tiefere Netze ([64], [128,64], [128,64,32], [128,64,32,16], [128,64,32,16] + Dropout). Pro Modell: Training über 300 Epochen, Loss-Kurven betrachtet, R^2 / MAE / RMSE notiert. (*ca. 4 h*)
- Dropout-Regularisierung eingebaut; Overfitting-Verhalten von Modell 3 vs. Modell 5 analysiert und im Notebook kommentiert. (*ca. 45 min*)
- **Zusammenfassungszelle in 05_Neural_Network.ipynb ausgefüllt:** Die zuvor mit „*eintragen*“ platzhalterten Felder in der Vergleichstabelle (Abschnitt 7.10) wurden mit den tatsächlichen Messwerten aus den Notebook-Outputs befüllt. (*ca. 30 min*)
- **Ergebnisse abgelesen** aus den Zell-Outputs der bereits ausgeführten Modellzellen:

Aspekt	Lineare Regression	Bestes NN ([128,64,32,16] + Dropout)
R^2 Test	0.6326	0.7795
MAE Test	0.4341	0.3066
Trainingszeit	< 1 s	~1–2 min (300 Epochs, CPU)

- **Fazit formuliert:** Das NN übertrifft die lineare Regression um ~18 Prozentpunkte R^2 und ~0.13 MAE (in 100.000 USD). Als Begründung wurde der nichtlineare Charakter des California-Housing-Datensatzes herausgearbeitet. Grenzen (Overfitting ohne Regularisierung, fehlende Interpretierbarkeit, hoher Tuning-Aufwand) sowie mögliche Verbesserungen (Early Stopping, LR-Scheduling, Ensembling) wurden dokumentiert. (*ca. 90 min*)
- Logbuch-Eintrag verfasst, Ergebnisse strukturiert zusammengefasst, Entscheidungen begründet. (*ca. 60 min*)

Entscheidungen

Entscheidung	Begründung
Modell 5 ([128,64,32,16] + Dropout) als bestes Modell	Höchster R^2 Test (0.7795) bei gleichzeitig geringster Generalisierungslücke dank Dropout
Overfitting-Vergleich Modell 3 vs. Modell 5 explizit erwähnt	Zeigt konkret den Nutzen von Dropout-Regularisierung

Schwierigkeiten

- Keine; Metrikwerte lagen bereits als Notebook-Output vor.

Eintrag 6 – Code-Review, Gruppentreffen 5 und Git-Support

Datum: 14.03.–17.03.2026 **Zeitraum:** Verteilt über mehrere Tage, Gruppentreffen am 17.03. (17:00–19:00 Uhr) **Gesamtzeit:** ca. 6 h

Arbeitsschritte

1. Code-Review und Beobachtung der Contributions (ca. 1,5 h)

- Zwischen dem 14. und 17. März kontinuierlich den Git-Commit-Verlauf beobachtet und neue Contributions der Teammitglieder verfolgt. Insgesamt wurden in diesem Zeitraum ca. 30–40 Commits von verschiedenen Teammitgliedern eingereicht. (ca. 30 min)
- **Felix H.:** Mehrere Commits zu seinem Neuronalen-Netz-Notebook (05_Neural_Network_Felix.ipynb): Feature-Selektion implementiert, Random Search mit 100 Modellen durchgeführt, Learning-Rate-Schedule integriert, verschiedene Ensemble-Methoden getestet. Bester Testscore: $R^2 = 0.7946$. Code durchgelesen und Ansätze nachvollzogen. (ca. 45 min)
- **Kolja:** Weiterarbeit an seinem NN-Notebook (05_Neural_Network_Kolja.ipynb), Training stabilisiert, Vergleich mit anderen Modellen ergänzt. Mehrere Logbuch-Einträge committed. (ca. 15 min)
- **Felix N. und Björn:** Initiale Logbücher angelegt (Logbuch_Felix_Neumann.md, Logbuch_Björn_Becker.md vermutlich). Beide haben begonnen, ihre Arbeit zu dokumentieren. (ca. 20 min)

2. Gruppentreffen 5 am 17.03. (17:00–19:00 Uhr) (ca. 2 h)

- Teilnahme am Gruppentreffen 5 via Discord. Alle fünf Teammitglieder anwesend (Florian, Kolja, Felix N., Felix H., Björn). (ca. 2 h)

- **Vorstellung der Zwischenergebnisse:** Kolja und Felix H. haben ihre bisherigen Ergebnisse zu den Neuronalen Netzen präsentiert. Felix H. hat seine Erkenntnisse zur Feature-Selektion vorgestellt: Manche Features tragen mehr Rauschen als Informationsgewinn bei; durch Reduktion auf wenige relevante Features konnte er bessere Ergebnisse erzielen. Außerdem wurde festgestellt, dass die Variation bei erneutem Trainieren sehr groß ist, was eine systematische Hyperparameter-Optimierung erschwert. *(in obigen 2 h enthalten)*
- **Aufgabenverteilung für die Endphase besprochen:** Person 1 soll Hyperparameter-Optimierung durchführen (Lernrate, Batch Size, Architektur, Regularisierung), Person 2 soll Modellvergleich mit alternativen Modellen (Random Forest, Gradient Boosting) durchführen. Björn und Felix N. wurden beauftragt, die bisherigen Ergebnisse zu konsolidieren und verständlich zu dokumentieren. *(in obigen 2 h enthalten)*
- **Organisatorisches:** Einigung auf Zitierstil (Harvard), LaTeX-Vorlage für Gruppenergebnisse soll erstellt werden, mindestens ein weiteres Treffen in KW 13 geplant (Datum noch offen). *(in obigen 2 h enthalten)*
- **GitHub-Probleme thematisiert:** Mehrere Teammitglieder berichteten von Schwierigkeiten mit Git-Befehlen (Push, Pull, Merge-Konflikte). Vereinfacht, dass ich nach dem Treffen Unterstützung leiste. *(in obigen 2 h enthalten)*

3. Git/GitHub-Support für Björn und Felix N. (ca. 2,5 h)

- **Björn:** Beim Gruppentreffen hat Björn Probleme beim Pushen seines Logbuchs gemeldet. Gemeinsam via Screensharing (Discord) das Problem analysiert: Lokale Änderungen waren nicht committed, außerdem lagen Remote-Änderungen vor, die noch nicht gepullt wurden. *(ca. 45 min)*
 - Schritt für Schritt durch den Workflow geführt: `git status` → `git add portfolio/Logbuch_Björn_Becker.md` → `git commit -m "Logbuch erstellt"` → `git pull --rebase` → `git push`. Push erfolgreich durchgeführt. *(in obigen 45 min enthalten)*
 - Zusätzlich erklärt, wie man Merge-Konflikte in VS Code löst (Konfliktmarker <<<<<<, =====, >>>>>> identifizieren, gewünschte Version auswählen oder manuell integrieren, dann `git add` und `git rebase --continue`). *(ca. 20 min)*
 - **Notebook-Output-Problem:** Björn hatte beim Comitten von Koljas Notebook (05_Neural_Network_Kolja.ipynb) bemerkt, dass die Datei über 1000 Zeilen Output enthielt, obwohl `nbstripout` eingerichtet war. Problem identifiziert: `nbstripout` war auf Björns System nicht als Git-Hook installiert. Gemeinsam `nbstripout --install` ausgeführt und mit einem Test-Commit verifiziert. Commit 01cb181 ("Problemlösung") reduzierte die Notebook-Datei von 1080 auf 71 Zeilen. *(ca. 30 min)*
- **Felix N.:** Felix N. hatte Schwierigkeiten beim Initialisieren seines Logbuchs und beim Pushen. Problem ähnlich wie bei Björn: Unklare Reihen-

folge von Add, Commit, Pull, Push. (*ca. 30 min*)

- Git-Workflow nochmals Schritt für Schritt erklärt: Lokale Änderungen → `git add` → `git commit` → `git pull` (um Remote-Änderungen zu holen) → `git push`. (*in obigen 30 min enthalten*)
- Visual Studio Code als Git-GUI vorgestellt: Source-Control-Tab gezeigt, Staging-Bereich erklärt, Commit-Message-Feld demonstriert. Felix N. hat dann eigenständig sein Logbuch committed und gepusht (Commit `be44740`: “initial logbook Felix_Neumann”). (*ca. 25 min*)
- **Felix H.:** Felix H. hatte am 14.03. zwei Merge-Konflikte gehabt (Commits `b75b0b4` und `84a2e01`: “Git Konflikt”). Nachträglich den Diff dieser Commits angeschaut: Konflikte in `05_Neural_Network_Felix.ipynb` und `Logbuch_Felix_Hollfoth.md`. Felix H. hatte die Konflikte bereits selbst gelöst, aber ich habe ihm nachträglich erklärt, wie man solche Konflikte vermeiden kann (häufiger pullen, kleinere Commits, Kommunikation im Team). (*ca. 20 min*)

Entscheidungen

Entscheidung	Begründung
Git-Support via Screensharing (Discord)	Ermöglicht direkte visuelle Anleitung und schnellere Problemlösung
VS Code als Git-GUI empfohlen	Niedrigere Einstiegshürde für Teammitglieder mit wenig Git-Erfahrung

Schwierigkeiten

- **Git-Workflow-Verständnis:** Björn und Felix N. hatten beide anfängliche Schwierigkeiten mit der korrekten Reihenfolge von Git-Befehlen (Add → Commit → Pull → Push). Beide haben jetzt ein grundlegendes Verständnis.
- **nbstripout nicht auf allen Systemen aktiv:** Kolja undd vermutlich Björn hatte `nbstripout` nicht als Git-Hook installiert, obwohl es in der `Ressources.md` dokumentiert war. Problem gelöst, aber zeigt, dass die initiale Setup-Dokumentation möglicherweise noch nicht von allen Teammitgliedern vollständig umgesetzt wurde.
- **Merge-Konflikte bei Felix H.:** Zwei Merge-Konflikte in einer Woche deuten darauf hin, dass die Team-Kommunikation über parallele Änderungen noch verbessert werden kann.

Nächste Schritte

- Weitere Unterstützung bei Git-Problemen anbieten, falls nötig.
- Aufbereitung der Ergebnisse im nächsten Gruppentreffen (KW 13) verfolgen.

- Prüfen, ob alle Teammitglieder `nbstripout` korrekt installiert haben.

Eintrag 7 – Nachtrag Gruppentreffen 4 und Neupriorisierung auf NN

Datum: 27.02.2026

Zeitraum: 15:00–17:15 Uhr (Gruppentreffen 4) + Nachbereitung

Gesamtzeit: ca. 3,5 h

Arbeitsschritte

- Teilnahme am Gruppentreffen 4 (Discord) mit allen Teammitgliedern; gemeinsamer Abgleich, dass wir in den vorherigen Schritten teils zu breit gearbeitet hatten und den Fokus klarer auf Neuronale Netze legen. *(ca. 2,25 h)*
- Ergebnisse zur bisherigen Datenbereinigung/Skalierung und zur Ordner- und Notebook-Struktur vorgestellt; abgestimmt, welche Teile als stabile Basis bestehen bleiben und welche Abschnitte für den NN-Fokus reduziert werden. *(ca. 25 min)*
- Technische Entscheidung zur Umgebung dokumentiert: Trotz TensorFlow-/Python-Problemen auf verschiedenen Systemen bleiben wir bei TensorFlow und akzeptieren zunächst inhomogene lokale Python-Versionen, solange Ergebnisse reproduzierbar dokumentiert werden. *(ca. 20 min)*
- Nach dem Treffen eigene Aufgaben neu priorisiert (weniger neue Baustellen, stärkerer Fokus auf Modellvergleich und Aufbereitung belastbarer NN-Ergebnisse für Portfolio/Logbuch). *(ca. 40 min)*

Entscheidungen

Entscheidung	Begründung
Fokusverschiebung auf Neuronale Netze	Passt besser zur Aufgabenstellung und zum bereits investierten Aufwand im Team
Heterogene Python-Setups zunächst akzeptieren	Praktische Lösung für unterschiedliche Systeme bei gleichzeitigem Zeitdruck

Schwierigkeiten

- Versionsthemen (TensorFlow/Python) blieben ein Risiko, konnten aber durch klare Dokumentation und reproduzierbare Notebook-Schritte abgefedert werden.

Eintrag 8 – Nachtrag Gruppentreffen 6 und Notebook-Outputs ohne nbstripout

Datum: 26.03.2026

Zeitraum: 17:30–19:30 Uhr (Gruppentreffen 6) + technische Umsetzung danach

Gesamtzeit: ca. 5 h

Arbeitsschritte

- Teilnahme am letzten Gruppentreffen (Discord): Vorstellung der Arbeiten von Felix N. und Björn, Abstimmung zur finalen Formatierung und Klärung der letzten Abgabeschritte. (*ca. 2 h*)
- Beschluss aus dem Treffen umgesetzt: Notebooks sollen in der finalen Fassung mit sichtbaren Zell-Outputs vorliegen, damit Ergebnisse für Korrektur und Nachvollziehbarkeit direkt erkennbar sind. (*ca. 20 min Planung*)
- **nbstripout** lokal deaktiviert (**nbstripout --uninstall**) und die relevanten Notebooks erneut vollständig ausgeführt, um Outputs konsistent zu speichern. Anschließend Result-Grafiken aktualisiert und mitgeprüft. (*ca. 2 h*)
- Änderungen in einem eigenen Commit dokumentiert: **2a0cfc2** (*“nbstripout deinstalliert, notebooks durchlaufen lassen. Output ist nun in allen ipynb notebooks abgespeichert!”*). Enthalten waren u. a. `0a_EDA.ipynb`, `0b_Baseline_Lineare_Regression.ipynb`, `05_Neural_Network_Florian.ipynb` sowie aktualisierte Ergebnisgrafiken. (*ca. 40 min inkl. Kontrolle*)

Entscheidungen

Entscheidung	Begründung
Outputs für finale Abgabe sichtbar machen	Bewertende sehen Metriken/Plots direkt im Notebook ohne erneute Ausführung
Eigener, klar benannter Commit für den Schritt	Erhöht Nachvollziehbarkeit im Git-Verlauf

Schwierigkeiten

- Ohne **nbstripout** steigen Dateigröße und Diff-Umfang der Notebooks deutlich; wurde bewusst zugunsten der Sichtbarkeit der Endergebnisse akzeptiert.

Eintrag 9 – Tau-Template bereitgestellt, Titelseite erstellt, Summary vollständig geschrieben

Datum: 28.03.–09.04.2026

Zeitraum: Mehrere Arbeitsblöcke im Selbststudium

Gesamtzeit: ca. 7 h

Arbeitsschritte

- Für die Portfolio-Zusammenfassungen ein geeignetes **Tau-Template** aus Overleaf/GitHub recherchiert, auf Projektkompatibilität geprüft (Struktur, Klassen-Dateien, Bibliographie, Build-Verhalten) und im Repository so abgelegt, dass alle Teammitglieder darauf zugreifen konnten. (*ca. 1,5 h*)
- Bei der Erstellung/Anpassung der Titelseite für die Summary unterstützt und Layout-/Metadatenfelder (Name, Matrikelnummer, Gruppe, Doctype, Datum) auf die Projektanforderungen abgestimmt. (*ca. 1,5 h*)
- Eigene vollständige Summary meiner Projektarbeit in `portfolio/Summary_Flo/main.tex` ausformuliert: Fragestellung, Methodik, EDA/Baseline, NN-Vergleich, Grenzen, Ausblick; inkl. Abbildungen/Tabellen und Verweisen auf den Arbeitsprozess. (*ca. 3 h*)
- Abschlusskorrekturen und Strukturverbesserungen durchgeführt (u. a. sprachliche Korrektur, Affiliation/Format, Literatur- und Build-Feinschliff), sodass die Summary konsistent kompilierbar und inhaltlich vollständig ist. (*ca. 1 h*)

Entscheidungen

Entscheidung	Begründung
Tau-Template als gemeinsame Grundlage	Einheitliche, professionelle Formatierung über alle Summaries hinweg
Eigene Summary komplett in LaTeX finalisiert	Saubere Referenzierung von Abbildungen/Tabellen und gute Nachvollziehbarkeit

Schwierigkeiten

- Kombination aus inhaltlicher Verdichtung (max. 4 Seiten) und formaler Qualität war aufwändig; erforderte mehrere Iterationen bei Textkürzung und Fokussierung.

Zeitübersicht (Gesamtprojekt)

Eintrag	Datum	Tätigkeit	Zeit
1	11.02.2026	Recherche Git-Workflow & nbstripout	ca. 4 h
2	17.02.2026	Repository- Strukturierung	ca. 3 h
3	20.–25.02.2026	Projektstruktur, Utils, Notebooks, EDA, Baseline	ca. 35 h
4	20.02.2026	Python- Versionskonflikt macOS	ca. 2 h
5	23.–26.02.2026	Neuronales Netz & Auswertung	ca. 11 h
6	14.–17.03.2026	Code-Review, Gruppentreffen 5, Git-Support	ca. 6 h
7	27.02.2026	Nachtrag Gruppentreffen 4 und Neupriorisierung	ca. 3,5 h
8	26.03.2026	Nachtrag Gruppentreffen 6 + Notebooks ohne nbstripout	ca. 5 h
9	28.03.–09.04.2026	Tau-Template, Titelseite, vollständige Summary	ca. 7 h
Gesamt			ca. 76,5 h